A08 · Diseño funcional: algoritmos, diagramas de flujo y casos de uso

Sistema de asistencia con reconocimiento facial · Secundaria Erich Fromm

Encabezado

Campo	Dato
Asignatura	Proyecto Integrador I (E-PIN1-1)
Unidad	III. Diseño, aplicación, desarrollo y pruebas del proyecto
Clave y nombre	A08 · Diseño funcional: algoritmos, diagramas de flujo y casos de uso
Modalidad	Equipo
Equipo	02
Integrantes	Lira Roacho Saulo Sebastian Páez Pérez Ángel Antonio Pérez Ramirez Emely Mayte Simental Muñoz Eliza Adelinne Rosales Meraz Ruth Paola
Proyecto	Sistema de asistencia con reconocimiento facial (Checador y Gestor de checadas)
Fecha	5 de noviembre de 2026

Contexto del sistema

El proyecto que estamos haciendo como equipo es un sistema de escritorio para la Secundaria Erich Fromm que sirve para gestionar las checadas de los docentes y del personal administrativo. Hoy en día la escuela checa con huella y, además, le piden entregar sus registros de asistencia en papel, por lo que la persona de recepción tiene que sacar toda esa información y dejarla lista para imprimir.

Para atender esto dividimos el sistema en dos partes que son del mismo proyecto:

- **Sistema 1: Checador.** Registra la entrada y la salida de cada empleado mediante un escáner facial. Lo usan los docentes y administrativos cuando llegan y cuando se van.
- **Sistema 2: Gestor de checadas.** Es el sistema con el que se “juega” con esos checados: mostrarlos, modificarlos, eliminar un registro y agregar un empleado nuevo. Solo lo puede usar la persona de recepción, que es la única con acceso a todas esas opciones. También permite exportar los checados a un archivo de Excel para imprimirlo.

Con esto la escuela sale beneficiada en dos cosas: el checado es más rápido porque el reconocimiento facial no obliga a poner el dedo en un lector como pasa con la huella, y la recepcionista ya no arma los reportes a mano porque el Excel se genera solo y se puede imprimir.

Nota de privacidad. Por el acuerdo con la dirección de la escuela, en el proyecto no se utiliza información personal real de los docentes ni su base de datos. Por eso todos los nombres, números de empleado, correos y horas que aparecen en este documento son datos ficticios de ejemplo.

Historias de usuario consideradas

Estas son las historias en las que nos basamos. HU-01 y HU-02 vienen de nuestro A05; HU-03 a HU-07 son historias nuevas que salieron al hacer el diseño y se agregan al Product Backlog. Además, en HU-02 cambiamos el formato del reporte de PDF a Excel, porque la recepcionista lo imprime desde ahí para entregar los registros en papel.

ID	Historia	Prioridad / origen
HU-01	Como docente, quiero posicionarme frente a la cámara, para registrar mi entrada (y mi salida) de forma automática.	Must (A05)
HU-02	Como administrador (recepcionista), quiero generar un reporte de asistencias en Excel, para entregarlo a la dirección sin usar listas de papel.	Must (A05, cambió de PDF a Excel)
HU-03	Como recepcionista, quiero iniciar sesión con mi usuario y contraseña, para que solo yo pueda usar el sistema de gestión.	Must (nueva)
HU-04	Como recepcionista, quiero agregar un empleado nuevo con su rostro, para que pueda checar en el sistema.	Must (nueva)
HU-05	Como recepcionista, quiero consultar los checados por rango de fechas, para revisar la asistencia del personal.	Must (nueva)
HU-06	Como recepcionista, quiero modificar un checado, para corregir errores en los registros.	Should (nueva)
HU-07	Como recepcionista, quiero eliminar un registro, para quitar checados equivocados o duplicados.	Should (nueva)

1. Diagrama de casos de uso

Identificamos dos actores. El **Empleado** solo interactúa con el Checador y el **Recepcionista** es quien usa el Gestor. El rectángulo es el límite del sistema; los óvalos son los casos de uso. Usamos <<include>> cuando un caso siempre necesita de otro (por ejemplo, registrar una

checada siempre verifica el rostro) y <<extend>> cuando agrega algo opcional (exportar a Excel solo se hace si antes se consultaron checados).

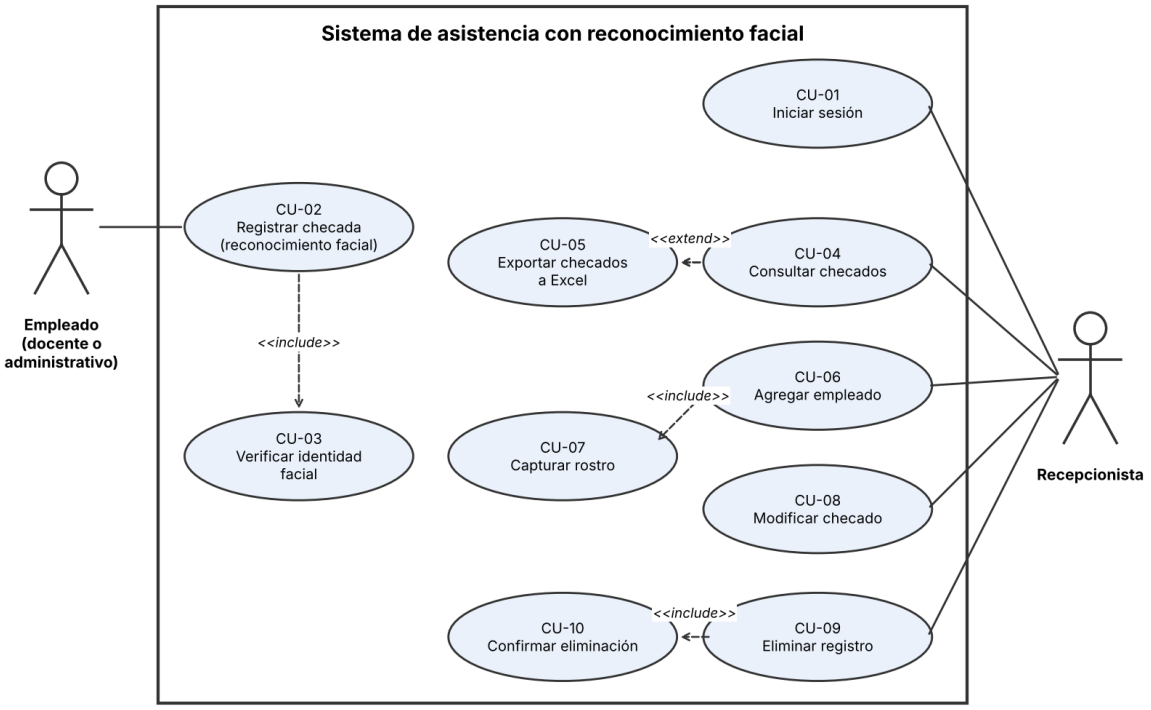


Figura 1. Diagrama de casos de uso del sistema de asistencia con reconocimiento facial.

Actor	Descripción	Casos de uso en los que participa
Empleado (docente o administrativo)	Personal de la secundaria que checa su entrada y salida frente a la cámara. No tiene acceso al Gestor.	CU-02 y, de forma incluida, CU-03
Recepcionista	Única persona con acceso al Gestor. Administra empleados y checados, y genera el Excel para imprimir.	CU-01, CU-04, CU-05, CU-06, CU-07, CU-08, CU-09 y CU-10

2. Especificación de casos de uso

Especificamos 5 casos de uso, los que están ligados a las historias Must. Los casos CU-08 y CU-09 (historias Should) quedan solo en el diagrama por ahora y se explican en la matriz de trazabilidad.

CU-01 · Iniciar sesión

Campo	Contenido
ID y nombre	CU-01 · Iniciar sesión
Actor(es)	Recepcionista

Campo	Contenido
Requisito / historia	RF-03 / HU-03
Precondiciones	El Gestor está abierto y la recepcionista tiene una cuenta creada.
Flujo básico	1. El sistema muestra la pantalla de inicio de sesión. 2. La recepcionista escribe su usuario y contraseña. 3. El sistema valida las credenciales. 4. El sistema abre el menú principal del Gestor.
Flujos alternos / de error	3a. Las credenciales son incorrectas: el sistema muestra “Datos incorrectos”, suma un intento y regresa al paso 2. 3b. Es el tercer intento fallido: el sistema bloquea la cuenta por 5 minutos y muestra un mensaje.
Postcondiciones	Hay una sesión activa y se muestran las opciones del Gestor (o la cuenta queda bloqueada temporalmente).
Reglas de negocio	RN-05, RN-06

CU-02 · Registrar checada con reconocimiento facial

Campo	Contenido
ID y nombre	CU-02 · Registrar checada con reconocimiento facial
Actor(es)	Empleado
Requisito / historia	RF-01 / HU-01
Precondiciones	El Checador está encendido, la cámara funciona y el empleado ya tiene su rostro registrado y su estatus es Activo.
Flujo básico	1. El empleado se para frente a la cámara. 2. El sistema captura la imagen del rostro. 3. El sistema verifica la identidad (incluye CU-03). 4. El sistema revisa que el empleado esté activo. 5. El sistema determina si la checada es de entrada o de salida. 6. El sistema guarda la checada con la fecha y la hora actuales. 7. El sistema muestra el tipo de checada registrada.
Flujos alternos / de error	3a. El rostro no se reconoce: se muestra “Rostro no reconocido” y regresa al paso 1. Si sigue fallando, el empleado acude con recepción. 4a. El empleado no está activo: se muestra “Empleado no activo” y no se guarda nada. 5a. Ya hay una checada del mismo empleado en los últimos 5 minutos: se muestra “Ya checaste hace unos minutos” y no se guarda otra.
Postcondiciones	La checada queda guardada en la base de datos con su fecha, hora y tipo.
Reglas de negocio	RN-01, RN-02, RN-03, RN-04

CU-04 · Consultar checados

Campo	Contenido
ID y nombre	CU-04 · Consultar checados
Actor(es)	Recepcionista
Requisito / historia	RF-02 / HU-05
Precondiciones	La recepcionista inició sesión (CU-01).
Flujo básico	1. La recepcionista elige “Consultar checados”. 2. Indica el rango de fechas y, si quiere, un empleado en específico. 3. El sistema valida que el rango sea correcto. 4. El sistema consulta la base de datos. 5. El sistema muestra los checados en una tabla.
Flujos alternos / de error	3a. La fecha de inicio es mayor que la fecha final: se muestra “Rango de fechas no válido” y regresa al paso 2. 4a. No hay checados en ese rango: se muestra “No hay checados en ese rango”.
Postcondiciones	Los checados se muestran en pantalla. No se modifica ningún dato.
Reglas de negocio	RN-10

CU-05 · Exportar checados a Excel

Campo	Contenido
ID y nombre	CU-05 · Exportar checados a Excel
Actor(es)	Recepcionista
Requisito / historia	RF-02 / HU-02
Precondiciones	Se hizo una consulta de checados (CU-04) con al menos un resultado. Este caso extiende a CU-04.
Flujo básico	1. La recepcionista presiona “Exportar a Excel”. 2. El sistema toma el rango de fechas de la consulta. 3. El sistema crea el archivo con sus encabezados. 4. Por cada registro escribe una fila y calcula las horas trabajadas. 5. El sistema guarda el archivo y avisa que se generó. 6. La recepcionista abre el archivo y lo imprime.
Flujos alternos / de error	2a. El rango de fechas no es válido o no hay registros: se muestra el mensaje correspondiente y no se crea el archivo. 4a. Un registro no tiene hora de salida: las horas se ponen en 0 y se anota “Sin salida”. 5a. No se puede guardar el archivo (por ejemplo, está abierto en otro programa): se avisa y se elige otro nombre.
Postcondiciones	Existe un archivo .xlsx listo para imprimir con los checados del rango.
Reglas de negocio	RN-10, RN-11, RN-12

CU-06 · Agregar empleado

Campo	Contenido
ID y nombre	CU-06 · Agregar empleado
Actor(es)	Recepcionista
Requisito / historia	RF-04 / HU-04
Precondiciones	La recepcionista inició sesión (CU-01) y el Gestor tiene acceso a la cámara.
Flujo básico	1. La recepcionista elige “Agregar empleado”. 2. Captura número de empleado, nombre, apellido(s), correo, teléfono y tipo (docente o administrativo). 3. El sistema valida los datos. 4. El sistema revisa que el número de empleado no exista. 5. El sistema captura el rostro del empleado (incluye CU-07). 6. El sistema guarda al empleado con estatus Activo. 7. El sistema muestra “Empleado registrado correctamente”.
Flujos alternos / de error	3a. Faltan datos obligatorios o el correo/teléfono no son válidos: se muestra el error, se cancela ese intento y la recepcionista vuelve al paso 2 para corregir y reenviar (el algoritmo se ejecuta de nuevo). 4a. El número de empleado ya existe: se muestra el error, se cancela ese intento y se vuelve al paso 2. 5a. No se logra capturar el rostro en 3 intentos: se cancela y el empleado no se guarda.
Postcondiciones	El empleado queda guardado con su rostro y ya puede checar en el Checador.
Reglas de negocio	RN-07, RN-08, RN-09

3. Reglas de negocio

ID	Regla	Caso de uso	Tipo
RN-01	Solo los empleados con estatus Activo pueden registrar una checada.	CU-02	Restricción
RN-02	La identidad solo se acepta si la coincidencia del rostro supera el umbral definido (80 %, valor tentativo que se ajustará en las pruebas).	CU-02, CU-03	Validación
RN-03	Las checadas de un empleado se alternan dentro del mismo día: si la última checada de hoy fue de entrada, la siguiente es de salida; si hoy no hay checadas o la última fue de salida, es de entrada.	CU-02	Cálculo
RN-04	No se acepta otra checada del mismo empleado dentro de los 5 minutos siguientes a la anterior (valor tentativo).	CU-02	Restricción
RN-05	Solo la recepcionista puede entrar al Gestor, con su usuario y contraseña.	CU-01	Permiso

ID	Regla	Caso de uso	Tipo
RN-06	Después de 3 intentos fallidos de inicio de sesión la cuenta se bloquea por 5 minutos.	CU-01	Restricción
RN-07	Nombre, primer apellido y tipo de empleado son obligatorios; el correo debe tener formato válido y el teléfono debe tener 10 dígitos.	CU-06	Validación
RN-08	El número de empleado no se puede repetir.	CU-06	Validación
RN-09	Un empleado nuevo no se guarda si no se capturó su rostro (máximo 3 intentos de captura).	CU-06, CU-07	Restricción
RN-10	La fecha de inicio no puede ser mayor que la fecha final al consultar o exportar.	CU-04, CU-05	Validación
RN-11	Horas trabajadas = hora de salida − hora de entrada. Si no hay salida, las horas son 0 y se anota “Sin salida”.	CU-05	Cálculo
RN-12	El Excel debe llevar estas columnas: número de empleado, nombre, tipo, fecha, hora de entrada, hora de salida, horas trabajadas y observación.	CU-05	Restricción
RN-13	Para modificar un checado se debe capturar el motivo y se guarda quién lo cambió y cuándo.	CU-08	Restricción
RN-14	Antes de eliminar un registro el sistema pide confirmación.	CU-09, CU-10	Restricción
RN-15	En el desarrollo y las pruebas no se usa información personal real de los docentes ni su base de datos (condición de la dirección).	General	Restricción

4. Algoritmos

Escribimos 4 algoritmos de la lógica de negocio directamente en Python con programación orientada a objetos (Visual Studio), que es el lenguaje y el paradigma con el que se va a desarrollar el sistema. Cada algoritmo es un método de la clase de su módulo (ver la estructura de clases más abajo). Todos usan decisiones y tres de ellos usan repeticiones (ALG-02, ALG-03 y ALG-04 con while). En los comentarios con # se indica la regla de negocio que cubre cada parte. Las líneas de class y __init__ solo arman la clase y sus dependencias; el algoritmo es el cuerpo del método, que sigue el mismo orden que su diagrama de flujo y su prueba de escritorio.

Estructura de clases (Python, POO)

Cada módulo de la sección 7 se convierte en una clase y cada algoritmo en un método de su clase. Las clases reciben sus dependencias por el constructor (por ejemplo, RegistroChecados recibe reconocedor y bd), así que ninguna crea ni conoce los detalles de otra, y eso mantiene el bajo acoplamiento. Los nombres de variables y métodos se escriben en snake_case, como recomienda PEP 8. La última columna relaciona cada clase con las pantallas del mockup en Figma: iniciar sesión, menú principal (Personal y Checados) y las pantallas de registrar,

actualizar, consultar, historial y eliminar. Solo los cuatro algoritmos de abajo se desarrollan completos en esta actividad; los demás métodos quedan para los siguientes sprints.

Clase	Módulo	Atributos principales	Métodos principales	Pantallas del mockup
Empleado	M5	num_emp, nombre, apellido_paterno, apellido_materno, puesto (tipo), codigo_postal, calle, colonia, correo, telefono, turno, estatus, especialidad, fecha_ingreso	Entidad: solo guarda datos.	Registrar, Actualizar y Consultar personal
Checada	M3, M6	id_checado, num_emp, fecha, hora, tipo ("Entrada" o "Salida"), observaciones	Entidad: solo guarda datos.	Registrar, Actualizar, Historial y Eliminar checado
RegistroDiario	M7	num_emp, nombre, tipo_empleado, fecha, hora_entrada, hora_salida	Entidad: junta la entrada y la salida de un empleado en un día (es lo que recorre ALG-04).	Historial de checados
CapturaFacial	M1	camara	capturar_rostro(num_emp)	Checador; Registrar personal
ReconocedorFacial	M2	umbral (RN-02)	reconocer_rostro(imagen_rostro)	Checador
RegistroChecados	M3	reconocedor, bd, interfaz	registrar_checada(imagen_rostro) → ALG-01	Checador
Autenticador	M4	bd, interfaz	iniciar_sesion() → ALG-02	Iniciar sesión
GestorEmpleados	M5	captura, bd, interfaz	agregar_empleado() → ALG-03 actualizar_empleado() consultar_empleado()	Registrar, Actualizar y Consultar personal
GestorChecados	M6	bd, interfaz	consultar_checados() modificar_checado() eliminar_checado()	Historial, Actualizar y Eliminar checado
ReporteExcel	M7	gestor_checados, interfaz	exportar_checados() → ALG-04	Historial de checados
BaseDatos	M8	conexion	estatus(), hay_checada_reciente(), ultimo_tipo_del_dia(), registrar_checado(), credenciales_correctas(), bloquear_cuenta(), existe_empleado(), guardar_empleado()	Ninguna (no tiene interfaz)
InterfazUsuario	M9	ventana principal	mostrar(), leer_credenciales(), leer_datos_empleado(), leer_rango_fechas(), abrir_menu_principal()	Todas las pantallas

ALG-01 · RegistrarChecada

Propósito	Identificar al empleado por su rostro y registrar si es una entrada o una salida (CU-02).
Entradas	imagen_rostro (imagen capturada por la cámara).
Salidas	Registro de la checada en la base de datos y un mensaje en pantalla.

```
class RegistroChecados:                          # M3
    def __init__(self, reconocedor, bd, interfaz):
        self.reconocedor = reconocedor           # M2
        self.bd = bd                             # M8
        self.interfaz = interfaz                 # M9

    def registrar_checada(self, imagen_rostro):
        # RN-02 (devuelve 0 si no coincide)
        num_emp = self.reconocedor.reconocer_rostro(imagen_rostro)
        if num_emp == 0:
            self.interfaz.mostrar("Rostro no reconocido")
            return

        # RN-01
        if self.bd.estatus(num_emp) != "Activo":
            self.interfaz.mostrar("Empleado no activo")
            return

        # RN-04
        if self.bd.hay_checada_reciente(num_emp, 5):
            self.interfaz.mostrar("Ya checaste hace unos minutos")
            return

        # RN-03
        if self.bd.ultimo_tipo_del_dia(num_emp) == "Entrada":
            tipo = "Salida"
        else:
            tipo = "Entrada"

        fecha_actual = date.today()
        hora_actual = datetime.now().time()
        self.bd.registrar_checado(num_emp, fecha_actual, hora_actual, tipo)
        self.interfaz.mostrar("Checada de " + tipo + " registrada")
```

ALG-02 · IniciarSesion

Propósito	Permitir el acceso al Gestor solo a la recepcionista y bloquear la cuenta tras 3 intentos fallidos (CU-01).
Entradas	usuario y contrasena.
Salidas	Menú principal del Gestor o mensaje de cuenta bloqueada.

```
class Autenticador:                              # M4
    def __init__(self, bd, interfaz):
        self.bd = bd                             # M8
        self.interfaz = interfaz                 # M9
```

```
    def iniciar_sesion(self):
        intentos = 0
        acceso = False
        while intentos < 3 and not acceso:
            usuario, contrasena = self.interfaz.leer_credenciales()
            # RN-05
            if self.bd.credenciales_correctas(usuario, contrasena):
                acceso = True
            else:
                intentos += 1
                self.interfaz.mostrar("Datos incorrectos")

        if acceso:
            self.interfaz.abrir_menu_principal()
        else:
            # RN-06
            self.bd.bloquear_cuenta(5)
            self.interfaz.mostrar("Cuenta bloqueada por 5 minutos")
```

ALG-03 · AgregarEmpleado

Propósito	Validar los datos de un empleado nuevo, capturar su rostro y guardarlo (CU-06 y CU-07).
Entradas	num_emp, nombre, apellido1, correo, telefono y tipo.
Salidas	Empleado guardado con estatus Activo o un mensaje de error.

```
class GestorEmpleados:                          # M5
    def __init__(self, captura, bd, interfaz):
        self.captura = captura                  # M1
        self.bd = bd                            # M8
        self.interfaz = interfaz                # M9

    def agregar_empleado(self):
        (num_emp, nombre, apellido1,
         correo, telefono, tipo) = self.interfaz.leer_datos_empleado()

        # RN-07
        if nombre == "" or apellido1 == "" or tipo == "":
            self.interfaz.mostrar("Faltan datos obligatorios")
            return

        # RN-07
        if not self.correo_valido(correo) or len(telefono) != 10:
            self.interfaz.mostrar("Correo o teléfono no válido")
            return

        # RN-08
        if self.bd.existe_empleado(num_emp):
            self.interfaz.mostrar("El número de empleado ya existe")
            return

        # RN-09
```

```
        rostro_capturado = False
        intentos = 0
        while not rostro_capturado and intentos < 3:
            rostro_capturado = self.captura.capturar_rostro(num_emp)
            if not rostro_capturado:
                intentos += 1

        if not rostro_capturado:
            self.interfaz.mostrar("No se pudo capturar el rostro")
            return

        self.bd.guardar_empleado(num_emp, nombre, apellido1,
                                 correo, telefono, tipo, "Activo")
        self.interfaz.mostrar("Empleado registrado correctamente")

    def correo_valido(self, correo):      # función de apoyo
        return re.fullmatch(r"[^@\s]+@[^@\s]+\.[^@\s]+", correo) is not None
```

ALG-04 · ExportarChecados

Propósito	Generar el archivo de Excel con los checados de un rango de fechas y calcular las horas trabajadas (CU-05).
Entradas	fecha_inicio y fecha_fin.
Salidas	Archivo .xlsx con una fila por empleado y día, o un mensaje de error.

```
class ReporteExcel:                              # M7
    def __init__(self, gestor_checados, interfaz):
        self.gestor_checados = gestor_checados    # M6
        self.interfaz = interfaz                  # M9

    def exportar_checados(self):
        fecha_inicio, fecha_fin = self.interfaz.leer_rango_fechas()

        # RN-10
        if fecha_inicio > fecha_fin:
            self.interfaz.mostrar("Rango de fechas no válido")
            return

        registros = self.gestor_checados.consultar_checados(
            fecha_inicio, fecha_fin)
        if len(registros) == 0:
            self.interfaz.mostrar("No hay checados en ese rango")
            return

        libro = self.crear_archivo_excel()
        self.escribir_encabezados(libro)      # RN-12
        i = 1
        while i <= len(registros):
            r = registros[i - 1]
            # RN-11
            if r.hora_salida is None:
                horas = 0
                observacion = "Sin salida"
```

```
            else:
                horas = r.hora_salida - r.hora_entrada
                observacion = ""
            self.escribir_fila(libro, r, horas, observacion)
            i += 1

        self.guardar_archivo(libro)
        self.interfaz.mostrar("Archivo generado")
```

5. Pruebas de escritorio

Cada algoritmo se ejecutó a mano con al menos un caso válido y uno que activa un error. Como lo pide la regla RN-15, todos los datos son ficticios.

ALG-01 · RegistrarChecada

Caso 1 (válido): empleado 1003 llega a las 07:02 y no tiene checadas previas

Paso	num_emp	estatus	tipo	Condición evaluada	Salida
1	1003	–	–	reconocer_rostro() devuelve 1003	–
2	1003	–	–	¿num_emp == 0? → Falso	–
3	1003	Activo	–	¿estatus != “Activo”? → Falso	–
4	1003	Activo	–	¿hay_checada_reciente? → Falso (no tiene checadas hoy)	–
5	1003	Activo	Entrada	¿ultimo_tipo_del_dia = “Entrada”? → Falso (hoy no hay checadas)	tipo = “Entrada”
6	1003	Activo	Entrada	registrar_checado(1003, 04/11/2026, 07:02, “Entrada”)	“Checada de Entrada registrada”

Caso 2 (error): rostro que no está registrado

Paso	num_emp	estatus	tipo	Condición evaluada	Salida
1	0	–	–	reconocer_rostro() devuelve 0	–
2	0	–	–	¿num_emp == 0? → Verdadero	“Rostro no reconocido” (termina)

Caso 3 (error): el empleado 1003 vuelve a checar a las 07:04

Paso	num_emp	estatus	tipo	Condición evaluada	Salida
1	1003	–	–	reconocer_rostro() devuelve 1003	–
2	1003	–	–	¿num_emp == 0? → Falso	–
3	1003	Activo	–	¿estatus != “Activo”? → Falso	–

Paso	num_emp	estatus	tipo	Condición evaluada	Salida
4	1003	Activo	–	¿hay_checada_reciente? → Verdadero (la anterior fue a las 07:02)	“Ya checaste hace unos minutos” (termina)

Se cumple RN-04: no se guarda una segunda checada.

Caso 4 (válido): el empleado 1003 olvidó checar su salida ayer y llega hoy a las 07:03

Paso	num_emp	estatus	tipo	Condición evaluada	Salida
1	1003	–	–	reconocer_rostro() devuelve 1003	–
2	1003	–	–	¿num_emp == 0? → Falso	–
3	1003	Activo	–	¿estatus != “Activo”? → Falso	–
4	1003	Activo	–	¿hay_checada_reciente? → Falso (la última fue ayer)	–
5	1003	Activo	Entrada	¿ultimo_tipo_del_dia = “Entrada”? → Falso (hoy no hay checadas; la entrada de ayer no cuenta)	tipo = “Entrada”
6	1003	Activo	Entrada	registrar_checado(1003, 05/11/2026, 07:03, “Entrada”)	“Checada de Entrada registrada”

Se cumple RN-03: el registro sin salida de ayer no cambia el tipo de la checada de hoy.

ALG-02 · IniciarSesion

Caso 1 (válido): usuario “recepcion” con la contraseña correcta

Paso	intentos	acceso	Condición evaluada	Salida
1	0	Falso	Se inicializan las variables	–
2	0	Falso	¿intentos < 3 y no acceso? → Verdadero; se leen usuario y contrasena	–
3	0	Verdadero	¿credenciales_correctas? → Verdadero	acceso = True
4	0	Verdadero	¿intentos < 3 y no acceso? → Falso (acceso ya es Verdadero)	Sale del ciclo
5	0	Verdadero	¿acceso? → Verdadero	abrir_menu_principal()

Caso 2 (error): tres contraseñas incorrectas seguidas

Paso	intentos	acceso	Condición evaluada	Salida
1	0	Falso	Se inicializan las variables	–
2	1	Falso	¿intentos < 3 y no acceso? → Verdadero; ¿credenciales_correctas? → Falso (contraseña “abc”)	“Datos incorrectos”

Paso	intentos	acceso	Condición evaluada	Salida
3	2	Falso	Ciclo → Verdadero; ¿credenciales_correctas? → Falso (contraseña “1234”)	“Datos incorrectos”
4	3	Falso	Ciclo → Verdadero; ¿credenciales_correctas? → Falso (contraseña “recepcion1”)	“Datos incorrectos”
5	3	Falso	¿intentos < 3 y no acceso? → Falso (intentos = 3)	Sale del ciclo
6	3	Falso	¿acceso? → Falso	bloquear_cuenta(5) y “Cuenta bloqueada por 5 minutos”

Se cumple RN-06: al tercer intento fallido se bloquea la cuenta.

ALG-03 · AgregarEmpleado

Caso 1 (válido): num_emp = 2001, Laura Martínez, laura.martinez@ejemplo.com, 6181234567, Docente

Paso	Condición evaluada / variables	Salida
1	Se leen los datos	–
2	¿nombre = “” O apellido1 = “” O tipo = “”? → Falso	–
3	¿no correo_valido(correo) o len(telefono) != 10? → Falso (el correo tiene @ y el teléfono 10 dígitos)	–
4	¿existe_empleado(2001)? → Falso	–
5	rostro_capturado = False, intentos = 0	–
6	¿no rostro_capturado y intentos < 3? → Verdadero; capturar_rostro() devuelve Verdadero	rostro_capturado = True
7	¿no rostro_capturado? → Falso (no se suma intento)	–
8	¿no rostro_capturado y intentos < 3? → Falso	Sale del ciclo
9	¿no rostro_capturado? → Falso	–
10	guardar_empleado(2001, “Laura”, “Martínez”, …, “Activo”)	“Empleado registrado correctamente”

Caso 2 (error): correo sin arroba y teléfono de 9 dígitos

Paso	Condición evaluada / variables	Salida
1	Se leen: num_emp = 2002, correo = “laura.martinez”, telefono = “618123456”	–

Paso	Condición evaluada / variables	Salida
2	¿nombre = “” O apellido1 = “” O tipo = “”? → Falso	–
3	¿no correo_valido(correo) o len(telefono) != 10? → Verdadero	“Correo o teléfono no válido” (termina)

Caso 3 (error): la cámara no logra capturar el rostro

Paso	Condición evaluada / variables	Salida
1	Los datos son válidos y existe_empleado(2003) → Falso; rostro_capturado = False, intentos = 0	–
2	Ciclo → Verdadero; capturar_rostro() devuelve Falso; intentos = 1	–
3	Ciclo → Verdadero; capturar_rostro() devuelve Falso; intentos = 2	–
4	Ciclo → Verdadero; capturar_rostro() devuelve Falso; intentos = 3	–
5	¿no rostro_capturado y intentos < 3? → Falso (intentos = 3)	Sale del ciclo
6	¿no rostro_capturado? → Verdadero	“No se pudo capturar el rostro” (termina)

Se cumple RN-09: el empleado no se guarda sin rostro.

ALG-04 · ExportarChecados

Caso 1 (válido): del 02/11/2026 al 04/11/2026 hay 3 registros

Paso	i	r	horas	Condición evaluada	Salida
1	–	–	–	Se leen las fechas; ¿fecha_inicio > fecha_fin? → Falso	–
2	–	–	–	consultar_checados() devuelve 3 registros; ¿len(registros) == 0? → Falso	–
3	1	–	–	Se crea el Excel y los encabezados; i = 1	–
4	1	1003 · 02/11 · 07:00 a 14:00	7:00	¿i <= 3? → Verdadero; ¿hora_salida is None? → Falso	Fila escrita; i = 2
5	2	1003 · 03/11 · 07:05 a 14:05	7:00	¿i <= 3? → Verdadero; ¿hora_salida is None? → Falso	Fila escrita; i = 3
6	3	2001 · 03/11 · 07:30 a (vacío)	0	¿i <= 3? → Verdadero; ¿hora_salida is None? → Verdadero	Fila con “Sin salida”; i = 4
7	4	–	–	¿i <= 3? → Falso	Sale del ciclo

Paso	i	r	horas	Condición evaluada	Salida
8	4	–	–	guardar_archivo()	“Archivo generado”

Caso 2 (error): fecha_inicio = 10/11/2026 y fecha_fin = 05/11/2026

Paso	i	r	horas	Condición evaluada	Salida
1	–	–	–	Se leen las fechas; ¿fecha_inicio > fecha_fin? → Verdadero	“Rango de fechas no válido” (termina)

Se cumple RN-10: no se genera ningún archivo.

Caso 3 (error): del 01/11/2026 al 01/11/2026 (domingo, sin checadas)

Paso	i	r	horas	Condición evaluada	Salida
1	–	–	–	¿fecha_inicio > fecha_fin? → Falso	–
2	–	–	–	consultar_checados() devuelve 0 registros; ¿len(registros) == 0? → Verdadero	“No hay checados en ese rango” (termina)

6. Diagramas de flujo

Hicimos un diagrama por algoritmo con la simbología ANSI/ISO 5807: óvalo para inicio y fin, rectángulo para procesos, rombo para decisiones, paralelogramo para entradas y salidas y flechas para el flujo. Los diagramas siguen el mismo orden que el código en Python, paso por paso, y usan los mismos nombres de variables y métodos.

ALG-01 · RegistrarChecada

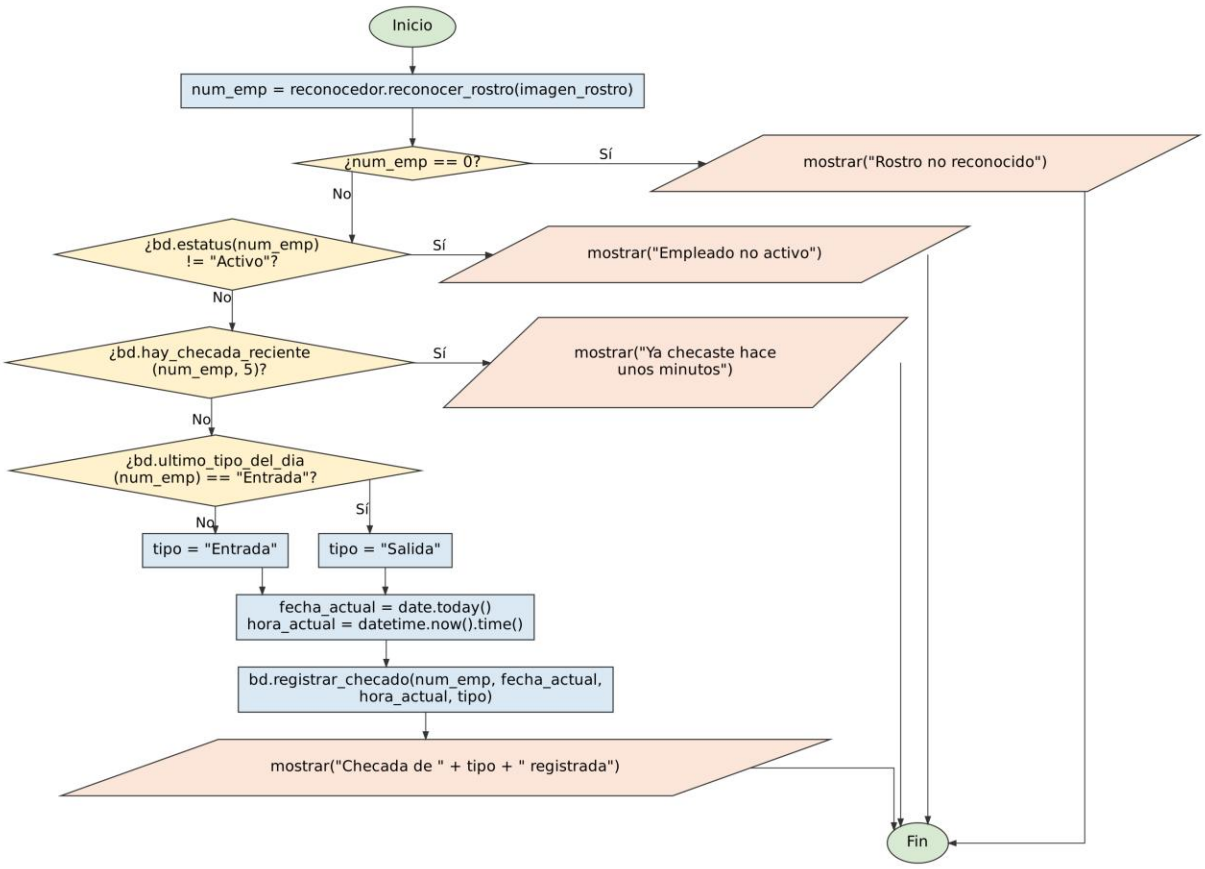


Figura 2. Diagrama de flujo de ALG-01 · RegistrarChecada.

ALG-02 · IniciarSesion

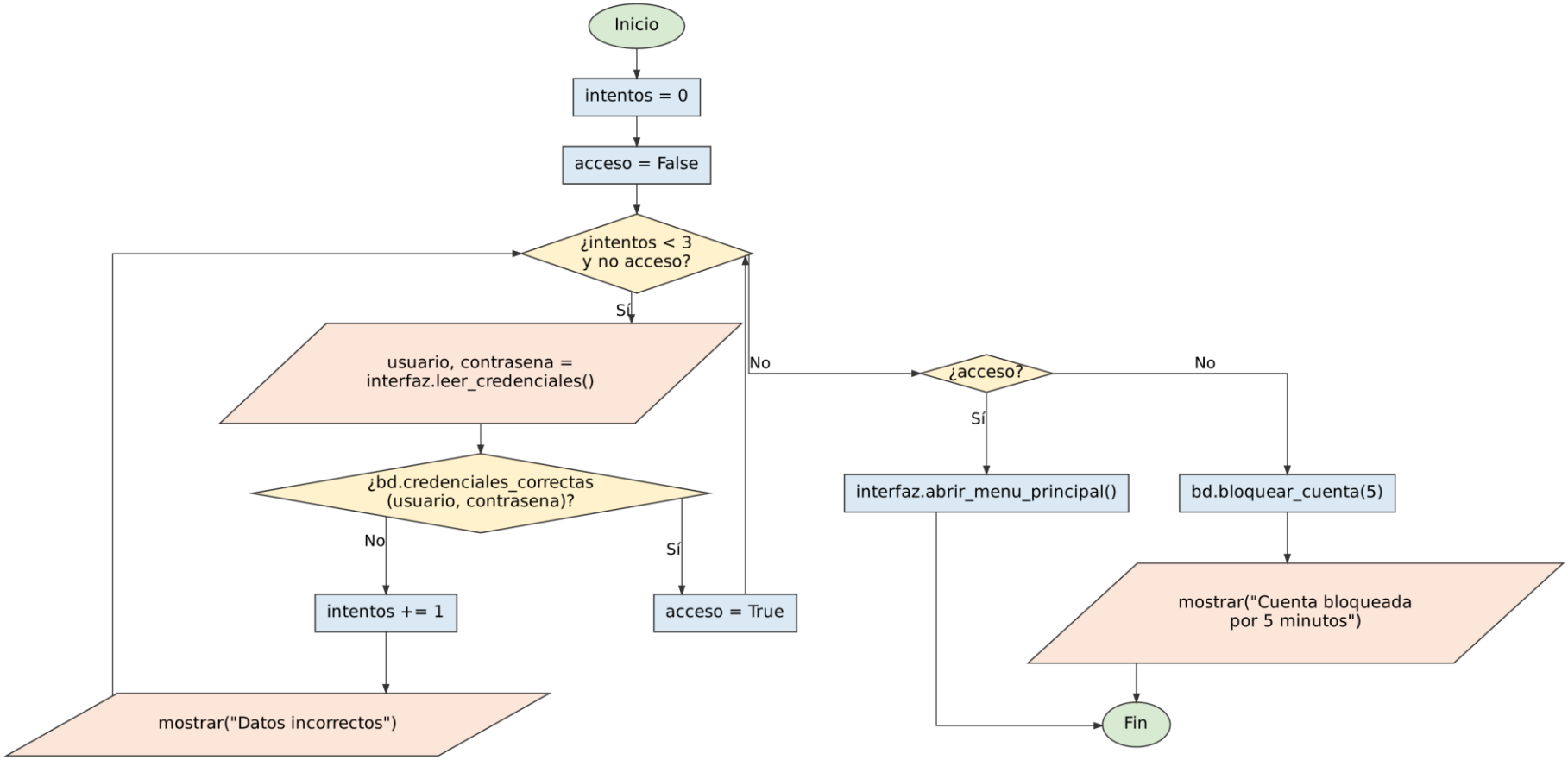


Figura 3. Diagrama de flujo de ALG-02 · IniciarSesion.

ALG-03 · AgregarEmpleado

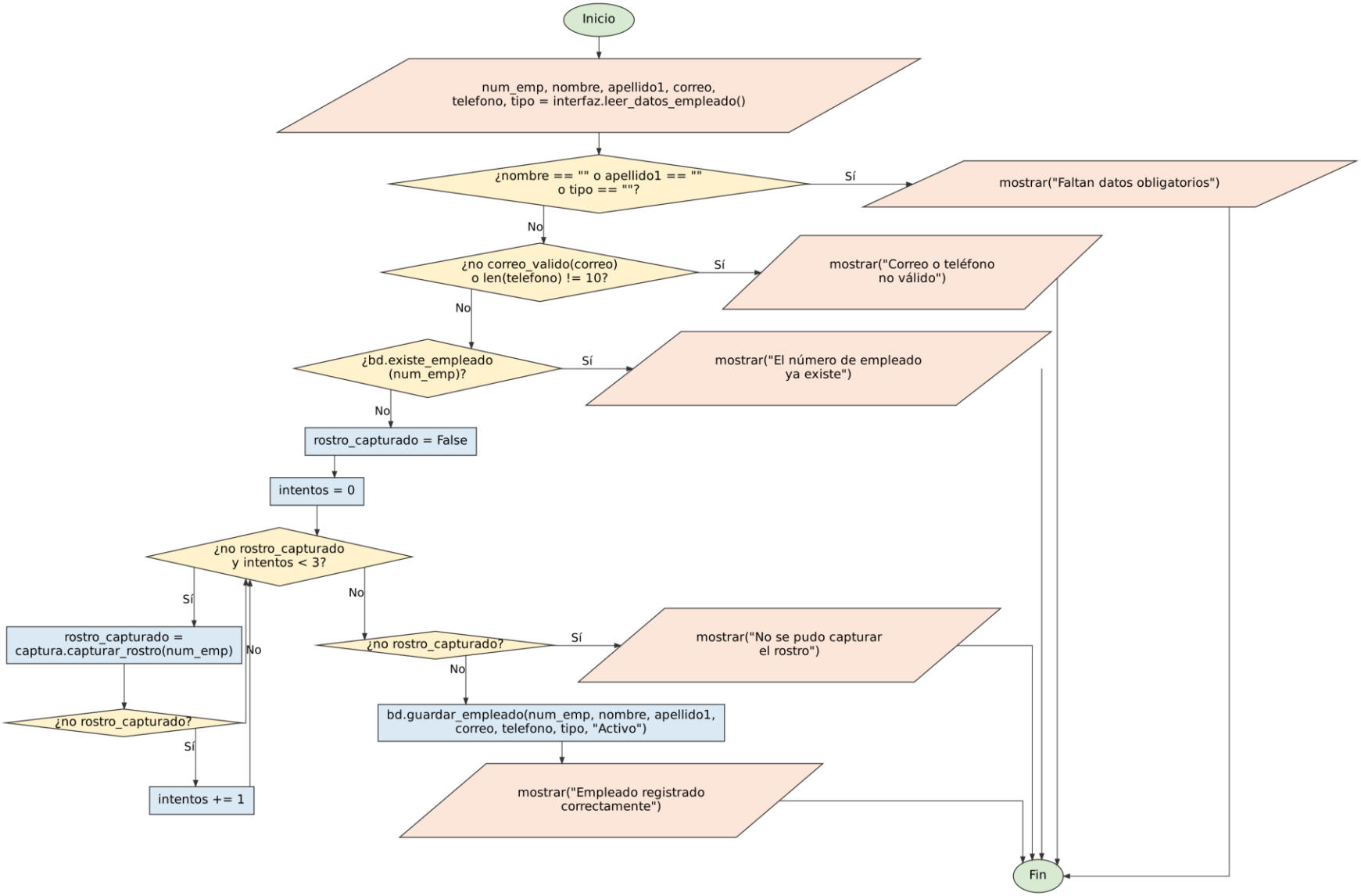


Figura 4. Diagrama de flujo de ALG-03 · AgregarEmpleado.

ALG-04 · ExportarChecados

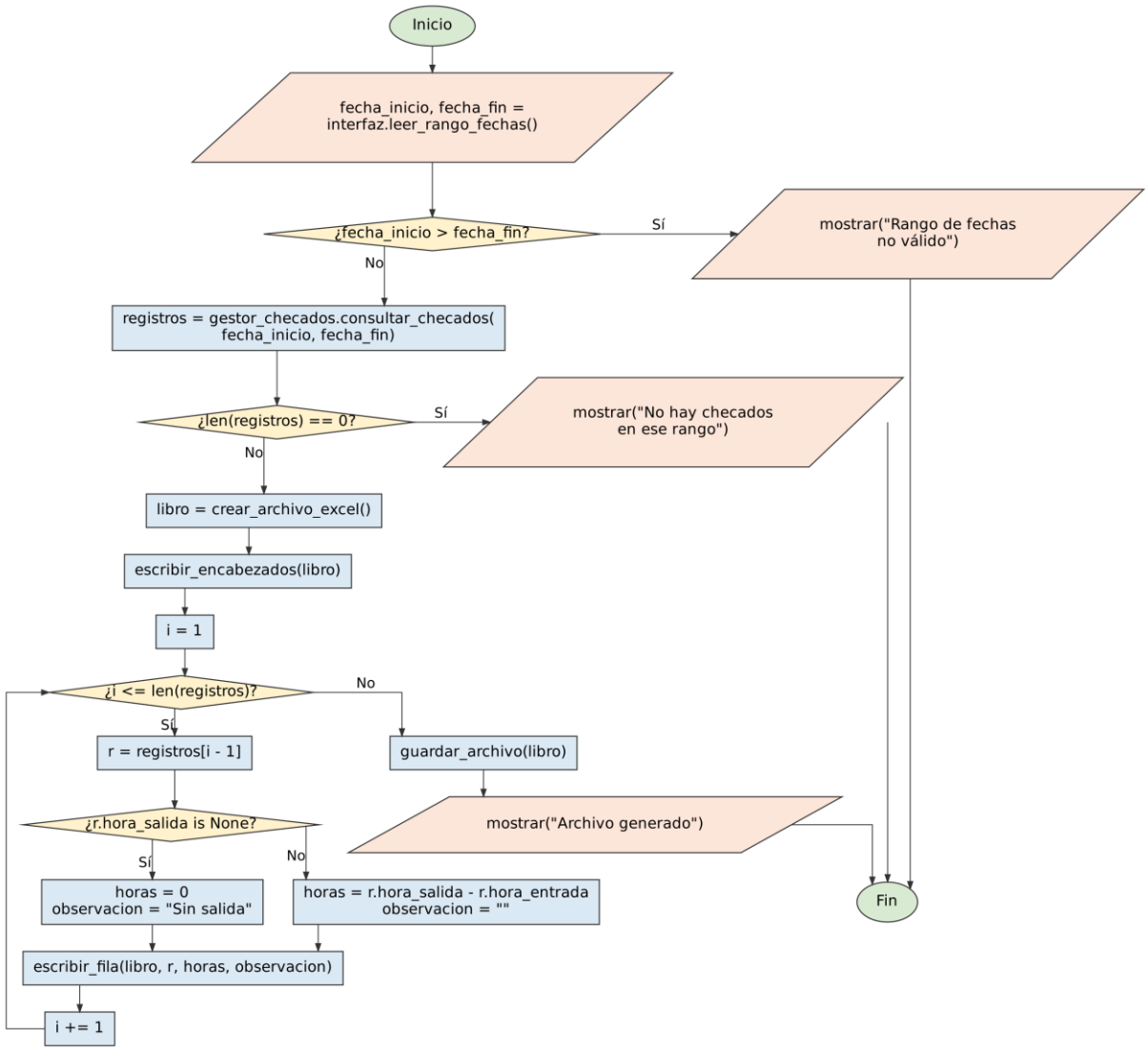


Figura 5. Diagrama de flujo de ALG-04 · ExportarChecados.

7. Módulos y principios de diseño funcional

Dividimos el sistema en módulos para que cada uno haga una sola cosa y para que el trabajo se pueda repartir entre los integrantes del equipo. Los primeros tres son del Checador y los demás del Gestor; el acceso a datos y la interfaz los usan los dos.

Módulo	Responsabilidad única	Depende de	Principio aplicado y justificación
M1 Captura facial	Obtener la imagen de la cámara y detectar que haya un rostro.	Cámara	Alta cohesión. Solo captura la imagen; no decide quién es la persona.

Módulo	Responsabilidad única	Depende de	Principio aplicado y justificación
M2 Reconocimiento facial	Comparar el rostro con los registrados y devolver el número de empleado (o 0).	M1, M8	Bajo acoplamiento. El Checador solo pregunta quién es y no necesita saber cómo se hace la comparación.
M3 Registro de checados	Decidir si es entrada o salida y guardar la checada.	M2, M8	Modularidad. Las reglas RN-01, RN-03 y RN-04 viven en un solo lugar.
M4 Autenticación	Validar usuario y contraseña, contar intentos y bloquear la cuenta.	M8	Separación de responsabilidades. Aquí se controla el permiso de la recepcionista (RN-05 y RN-06).
M5 Gestión de empleados	Agregar, consultar y eliminar empleados.	M1, M8	Modularidad. Reutiliza M1 para capturar el rostro en lugar de repetir ese código.
M6 Gestión de checados	Consultar, modificar y eliminar checados.	M8	Alta cohesión. Todo lo que se hace con un checado ya guardado está junto.
M7 Reportes	Generar el archivo de Excel con las horas trabajadas.	M6, M8	Separación de responsabilidades. Los cálculos del reporte no se mezclan con las pantallas.
M8 Acceso a datos	Conectarse a la base de datos y ejecutar las consultas.	Base de datos	Separación de responsabilidades. Es el único módulo que toca la base de datos; si algo cambia ahí, solo se modifica este módulo.
M9 Interfaz de usuario	Mostrar las pantallas del Checador y del Gestor y recibir los datos.	M3 a M7	Separación de responsabilidades. Las pantallas no tienen reglas de negocio, solo llaman a los módulos.

En resumen, aplicamos la **modularidad** al dividir el sistema en 9 partes, la **cohesión** al darle a cada módulo una sola función, el **bajo acoplamiento** al hacer que los módulos se comuniquen solo por lo necesario (por ejemplo, ningún módulo ejecuta consultas a la base de datos por su cuenta) y la **separación de responsabilidades** al dejar aparte la interfaz, la lógica y los datos.

8. Matriz de trazabilidad

Requisito	Historia de usuario	Caso de uso	Algoritmo / diagrama	Observaciones
RF-01	HU-01 Checar con el rostro	CU-02, CU-03	ALG-01 / Figura 2	Requisito de A05. Completo.
RF-02	HU-02 Reporte en Excel, HU-05 Consultar checados	CU-05, CU-04	ALG-04 / Figura 5 (la consulta usa consultar_checados())	Requisito de A05 (ahí el reporte era PDF). El Excel está diseñado; la consulta tiene caso de uso pero no un algoritmo propio, se diseñará en el siguiente sprint.
RF-03	HU-03 Iniciar sesión	CU-01	ALG-02 / Figura 3	Requisito nuevo surgido en el diseño. Completo.

Requisito	Historia de usuario	Caso de uso	Algoritmo / diagrama	Observaciones
RF-04	HU-04 Agregar empleado	CU-06, CU-07	ALG-03 / Figura 4	Requisito nuevo surgido en el diseño. Completo.
RF-05	HU-06 Modificar checado	CU-08	Pendiente	Hueco: es historia Should y por tiempo no se especificó. Ya existe la regla RN-13; falta su caso de uso detallado y su algoritmo.
RF-06	HU-07 Eliminar registro	CU-09, CU-10	Pendiente	Hueco: igual que el anterior, ya existe RN-14 y falta el algoritmo.
RNF-01	(aplica a todas las historias)	Todos	Módulo M9 (interfaz)	Requisito de A05 (navegación limpia para usuarios no técnicos). Se cubre en el diseño de las pantallas, no tiene algoritmo.

9. Revisión entre pares

Hallazgo del otro equipo	Elemento afectado	Corrección aplicada
En ALG-01, la función ultimo_tipo(num_emp) no se limita al día: si un empleado olvida checar su salida, al día siguiente su primera checada se registraría como “Salida”.	ALG-01, CU-02 y RN-03	Se cambió por ultimo_tipo_del_dia(num_emp), que solo considera las checadas de hoy. Se actualizaron RN-03, el código en Python, el diagrama de flujo (Figura 2) y se agregó el caso 4 a la prueba de escritorio.
En ALG-02 solo se muestra el mensaje “Cuenta bloqueada”, pero no hay ninguna acción que bloquee la cuenta de verdad.	ALG-02 y RN-06	Se agregó el paso bloquear_cuenta(5) antes del mensaje, en el código en Python y en el diagrama de flujo (Figura 3). La prueba de escritorio del caso 2 también se actualizó.
El caso de uso CU-06 dice que ante datos inválidos “regresa al paso 2”, pero el algoritmo ALG-03 termina; no coinciden.	CU-06 y ALG-03	Se aclararon los flujos alternos 3a y 4a de CU-06: se cancela el intento y la recepcionista vuelve al paso 2 y reenvía el formulario, con lo que el algoritmo se ejecuta de nuevo.

10. Referencias y declaración de uso de IA

Referencias

Gazoni, E., & Clark, C. (s. f.). openpyxl: A Python library to read/write Excel 2010 xlsx/xlsm files. https://openpyxl.readthedocs.io

Martin, R. C. (2008). Clean code: A handbook of agile software craftsmanship. Prentice Hall.

Object Management Group. (2017). Unified Modeling Language (UML), versión 2.5.1. https://www.omg.org/spec/UML/2.5.1

Stevens, W. P., Myers, G. J., & Constantine, L. L. (1974). Structured design. IBM Systems Journal, 13(2), 115–139. https://doi.org/10.1147/sj.132.0115

van Rossum, G., & Python development team. (s. f.). The Python tutorial. Python Software Foundation. https://docs.python.org/3/tutorial/

van Rossum, G., Warsaw, B., & Coghlan, A. (2001). PEP 8 – Style guide for Python code. Python Software Foundation. https://peps.python.org/pep-0008/

Declaración de uso de IA

La herramienta que se uso fue Claude para corrección de ortografía y modificación de oraciones para quitar redundancias.